

Celebal SQL Assignment 2

ecommerce_sales Databas

Name: Manas Arora

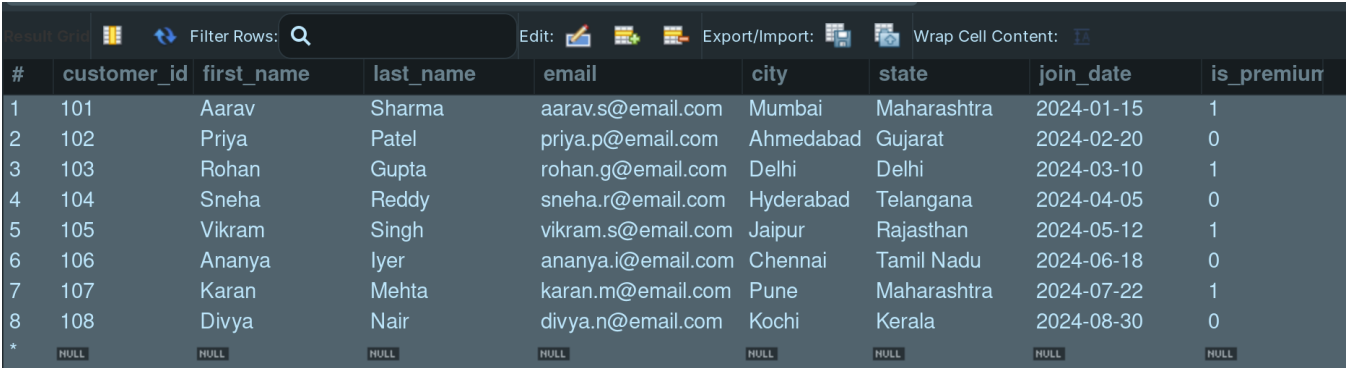
Intern ID: CT_CSI_DE_1089

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customers table.

```
| SELECT * FROM customers;
```

Screenshot:



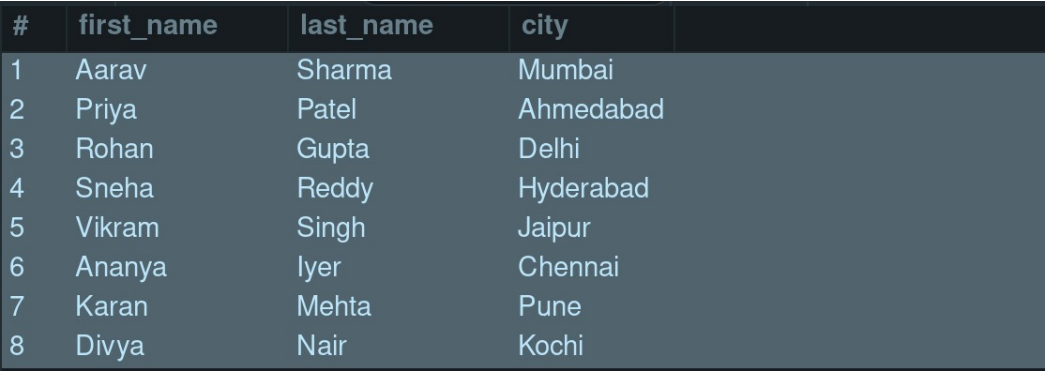
The screenshot shows a database interface with a query result grid. The query executed is 'SELECT * FROM customers;'. The result grid displays 9 columns: #, customer_id, first_name, last_name, email, city, state, join_date, and is_premium. There are 8 rows of data, each representing a customer. The last row is marked with an asterisk and 'NULL' for the is_premium column.

#	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

```
| SELECT first_name, last_name, city FROM customers;
```

Screenshot:



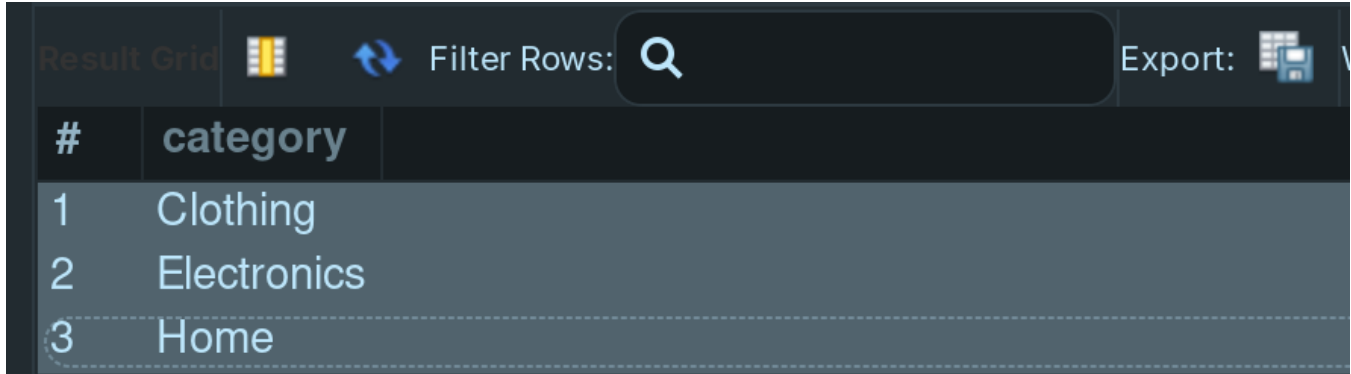
The screenshot shows a database query result grid. The query executed is 'SELECT first_name, last_name, city FROM customers;'. The result grid displays 4 columns: #, first_name, last_name, and city. There are 8 rows of data, each representing a customer.

#	first_name	last_name	city
1	Aarav	Sharma	Mumbai
2	Priya	Patel	Ahmedabad
3	Rohan	Gupta	Delhi
4	Sneha	Reddy	Hyderabad
5	Vikram	Singh	Jaipur
6	Ananya	Iyer	Chennai
7	Karan	Mehta	Pune
8	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
| SELECT DISTINCT category FROM products;
```

Screenshot:



The screenshot shows a database interface with a 'Result Grid' tab. It includes a 'Filter Rows' search bar and an 'Export' button. The query result is displayed in a table with two columns: '#' and 'category'. The results are as follows:

#	category
1	Clothing
2	Electronics
3	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

```
-- Primary Keys:
-- customers    → customer_id (INT)
-- products     → product_id  (INT)
-- orders       → order_id    (INT)
-- order_items  → item_id     (INT)

-- A Primary Key uniquely identifies each row in a table.
-- It must be UNIQUE so no two rows share the same identifier.
-- It must be NOT NULL because NULL means 'unknown',
-- and you cannot uniquely identify a row with an unknown value.
-- Together these guarantee every row can be referenced exactly.
```

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

```
-- Constraints on email: NOT NULL, UNIQUE

-- Attempting to insert a duplicate email:
INSERT INTO customers VALUES
(109, 'Test', 'User', 'aarav.s@email.com', 'Mumbai', 'Maharashtra', '2024-09-01',
FALSE);

-- Result: ERROR - Duplicate entry 'aarav.s@email.com' for key 'customers.email'
-- MySQL enforces the UNIQUE constraint and rejects the INSERT.
```

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write the INSERT statement and explain the error.

```
INSERT INTO products VALUES
(209, 'Broken Item', 'Electronics', 'TestBrand', -50.00, 100);

-- Result: ERROR - Check constraint 'products_chk_1' is violated.
-- The column has: CHECK (unit_price > 0)
-- A negative price violates this CHECK constraint, so MySQL rejects the row.
```

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

```
SELECT * FROM orders WHERE status = 'Delivered';
```

Screenshot:

#	order_id	customer_id	order_date	status	total_amount	
1	1001	101	2024-08-01	Delivered	4498.00	
2	1002	102	2024-08-03	Delivered	799.00	
3	1004	101	2024-08-10	Delivered	3499.00	
4	1006	105	2024-08-15	Delivered	5898.00	
5	1008	103	2024-08-20	Delivered	899.00	
6	1010	108	2024-08-28	Delivered	1598.00	
*	NULL	NULL	NULL	NULL	NULL	

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

```
SELECT * FROM products
WHERE category = 'Electronics' AND unit_price > 2000;
```

Screenshot:

#	product_id	product_name	category	brand	unit_price	stock_qty	
1	205	Bluetooth Speaker	Electronics	JBL	3499.00	200	
2	203	Smart Watch	Electronics	Noise	2999.00	150	
*	NULL	NULL	NULL	NULL	NULL	NULL	

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
SELECT * FROM customers
WHERE YEAR(join_date) = 2024 AND state = 'Maharashtra';
```

Screenshot:

#	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
SELECT * FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status <> 'Cancelled';
```

Screenshot:

#	order_id	customer_id	order_date	status	total_amount
1	1004	101	2024-08-10	Delivered	3499.00
2	1006	105	2024-08-15	Delivered	5898.00
3	1007	106	2024-08-18	Pending	1299.00
4	1008	103	2024-08-20	Delivered	899.00
5	1009	107	2024-08-25	Shipped	6098.00
*	NULL	NULL	NULL	NULL	NULL

Q11. Explain what idx_orders_date does and how it improves performance. Write a sample query that benefits from this index.

```
-- idx_orders_date is a B-Tree index on orders(order_date).
-- Without it, MySQL scans ALL rows to find matching dates (full table scan).
-- With it, MySQL jumps directly to the relevant date range in O(log n) time.

-- Sample query that benefits from idx_orders_date:
SELECT * FROM orders
WHERE order_date BETWEEN '2024-08-01' AND '2024-08-15';
```

Q12. If you run `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Rewrite to be SARGable.

```
-- YEAR(join_date) wraps the column in a function, breaking index usage.
-- MySQL cannot use the index because it must compute YEAR() for every row.

-- SARGable rewrite:
SELECT * FROM customers
WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01';
```

Screenshot:

#	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

```
SELECT COUNT(*) AS total_orders FROM orders;
```

Screenshot:

#	total_orders
1	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

```
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';
```

Screenshot:

The screenshot shows a database interface with a 'Result Grid' header. Below the header, there is a search bar labeled 'Filter Rows:' and buttons for 'Export' and 'Wrap Cell Content:'. The data table has two columns: '#' and 'total_revenue'. The first row shows the value '1' in the '#' column and '17191.00' in the 'total_revenue' column.

#	total_revenue
1	17191.00

Q15. Calculate the average unit_price of products in each category.

```
SELECT category, AVG(unit_price) AS avg_price
FROM products
GROUP BY category;
```

Screenshot:

The screenshot shows a database interface with a 'Result Grid' header. Below the header, there is a search bar labeled 'Filter Rows:' and buttons for 'Export' and 'Wrap Cell Content:'. The data table has three columns: '#', 'category', and 'avg_price'. The first three rows show the following data: (1, Clothing, 2699.000000), (2, Electronics, 2224.000000), and (3, Home, 949.000000).

#	category	avg_price
1	Clothing	2699.000000
2	Electronics	2224.000000
3	Home	949.000000

Q16. For each order status, find the count of orders and total revenue. Sort by total revenue descending.

```
SELECT status,
       COUNT(*) AS order_count,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Screenshot:

The screenshot shows a database interface with a 'Result Grid' header. Below the header, there is a search bar labeled 'Filter Rows:' and buttons for 'Export' and 'Wrap Cell Content:'. The data table has four columns: '#', 'status', 'order_count', and 'total_revenue'. The first four rows show the following data: (1, Delivered, 6, 17191.00), (2, Shipped, 2, 13596.00), (3, Cancelled, 1, 2999.00), and (4, Pending, 1, 1299.00).

#	status	order_count	total_revenue
1	Delivered	6	17191.00
2	Shipped	2	13596.00
3	Cancelled	1	2999.00
4	Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```
SELECT category,
       MAX(unit_price) AS max_price,
       MIN(unit_price) AS min_price
FROM products
GROUP BY category;
```

Screenshot:

#	category	max_price	min_price
1	Clothing	4599.00	799.00
2	Electronics	3499.00	899.00
3	Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000.

```
SELECT category, AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Screenshot:

#	category	avg_price
1	Clothing	2699.000000
2	Electronics	2224.000000

Section D — Joins & Relationships

Q19. INNER JOIN: Display each order with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT o.order_id, o.order_date,
       c.first_name, c.last_name,
       o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Screenshot:

Result Grid



Filter Rows:



Export:



Wrap Cell Content:



#	order_id	order_date	first_name	last_name	total_ammoun	
1	1001	2024-08-01	Aarav	Sharma	4498.00	
2	1004	2024-08-10	Aarav	Sharma	3499.00	
3	1002	2024-08-03	Priya	Patel	799.00	
4	1003	2024-08-05	Rohan	Gupta	7498.00	
5	1008	2024-08-20	Rohan	Gupta	899.00	
6	1005	2024-08-12	Sneha	Reddy	2999.00	
7	1006	2024-08-15	Vikram	Singh	5898.00	
8	1007	2024-08-18	Ananya	Iyer	1299.00	
9	1009	2024-08-25	Karan	Mehta	6098.00	
10	1010	2024-08-28	Divya	Nair	1598.00	

Q20. LEFT JOIN: List ALL customers and their orders (if any). Customers with no orders show NULL for order columns.

```
SELECT c.customer_id, c.first_name, c.last_name,
       o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Screenshot:

Result Grid		Filter Rows:		Export:		Wrap Cell Content:	
#	customer_id	first_name	last_name	order_id	order_date	total_ammoun	
1	101	Aarav	Sharma	1001	2024-08-01	4498.00	
2	101	Aarav	Sharma	1004	2024-08-10	3499.00	
3	102	Priya	Patel	1002	2024-08-03	799.00	
4	103	Rohan	Gupta	1003	2024-08-05	7498.00	
5	103	Rohan	Gupta	1008	2024-08-20	899.00	
6	104	Sneha	Reddy	1005	2024-08-12	2999.00	
7	105	Vikram	Singh	1006	2024-08-15	5898.00	
8	106	Ananya	Iyer	1007	2024-08-18	1299.00	
9	107	Karan	Mehta	1009	2024-08-25	6098.00	
10	108	Divya	Nair	1010	2024-08-28	1598.00	

Q21. Join orders → order_items → products to show: order_id, product_name, quantity, unit_price, discount_pct.

```
SELECT o.order_id,
       p.product_name,
       oi.quantity,
       oi.unit_price,
       oi.discount_pct
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p    ON oi.product_id = p.product_id;
```

Screenshot:

#	order_id	product_name	quantity	unit_price	discount_pct
1	1001	Wireless Earbuds	2	1499.00	0.00
2	1006	Wireless Earbuds	1	1499.00	10.00
3	1002	Cotton T-Shirt	1	799.00	0.00
4	1003	Smart Watch	1	2999.00	0.00
5	1005	Smart Watch	1	2999.00	0.00
6	1003	Running Shoes	1	4599.00	5.00
7	1006	Running Shoes	1	4599.00	5.00
8	1004	Bluetooth Speaker	1	3499.00	0.00
9	1009	Bluetooth Speaker	1	3499.00	0.00
10	1007	Bedsheet Set	1	1299.00	0.00
11	1010	Bedsheet Set	1	1299.00	0.00
12	1001	Laptop Stand	1	899.00	10.00
13	1008	Laptop Stand	1	899.00	0.00
14	1009	Cushion Covers...	2	599.00	15.00
15	1010	Cushion Covers...	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN. When would you use a FULL OUTER JOIN?

```
-- Example LEFT JOIN (all customers, even those without orders):
SELECT c.first_name, o.order_id
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

SELECT c.first_name, o.order_id
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.first_name, o.order_id
FROM customers c RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

#	first_name	order_id
1	Aarav	1001
2	Aarav	1004
3	Priya	1002
4	Rohan	1003
5	Rohan	1008
6	Sneha	1005
7	Vikram	1006
8	Ananya	1007
9	Karan	1009
10	Divya	1010

Q23. Identify all Foreign Key relationships. What happens if you insert an order with customer_id = 999?

```
-- Foreign Key relationships:
-- orders.customer_id      → customers.customer_id
-- order_items.order_id    → orders.order_id
-- order_items.product_id  → products.product_id

-- Attempting to insert with non-existent customer_id:
INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 500.00);

-- Result: ERROR - Cannot add or update a child row:
-- a foreign key constraint fails (orders.customer_id → customers.customer_id)
-- MySQL enforces Referential Integrity and rejects the INSERT.
```

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Use CASE to classify products into price tiers: Budget (<1000), Mid-Range (1000–3000), Premium (>3000).

```
SELECT product_name,
       unit_price,
       CASE
         WHEN unit_price < 1000 THEN 'Budget'
         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
         ELSE 'Premium'
       END AS price_tier
FROM products;
```

Screenshot:

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
#	product_name	unit_price	price_tier	
1	Wireless Earbuds	1499.00	Mid-Range	
2	Cotton T-Shirt	799.00	Budget	
3	Smart Watch	2999.00	Mid-Range	
4	Running Shoes	4599.00	Premium	
5	Bluetooth Speaker	3499.00	Premium	
6	Bedsheet Set	1299.00	Mid-Range	
7	Laptop Stand	899.00	Budget	
8	Cushion Covers...	599.00	Budget	

Q25. Using CASE inside an aggregate, count 'Delivered' vs 'Not Delivered' orders in a single row.

```
SELECT
  SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered,
  SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered
FROM orders;
```

Screenshot:

#	delivered	not_delivered
1	6	4

Q26. Explain each letter of ACID with a real-world bank transfer example.-- A – Atomicity:-- All operations in a transaction succeed or ALL are rolled back.

```
-- Bank transfer: debit ₹1000 AND credit ₹1000 must both happen;
-- if credit fails, debit is also reversed.

-- C - Consistency:
-- A transaction brings the DB from one valid state to another.
-- Bank transfer: total money in the system stays the same before and after.

-- I - Isolation:
-- Concurrent transactions don't interfere with each other.
-- Bank transfer: another user can't see the intermediate state where
-- money was debited but not yet credited.

-- D - Durability:
-- Once committed, changes survive crashes or power failures.
-- Bank transfer: after COMMIT, the credited ₹1000 is permanently saved
-- even if the server restarts immediately after.
```

Q27. Write a SQL transaction: insert order 1011, insert 2 order items, update stock_qty. ROLLBACK on failure, else COMMIT.

```
START TRANSACTION;

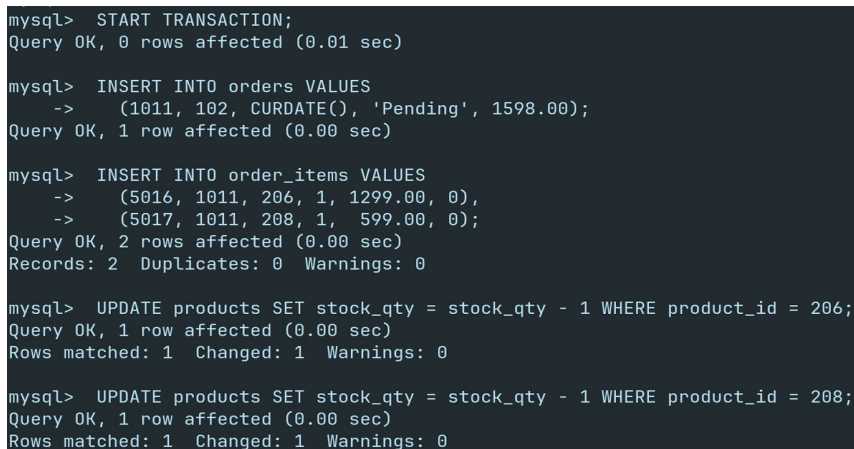
-- Step 1: Insert the new order
INSERT INTO orders VALUES
  (1011, 102, CURDATE(), 'Pending', 1598.00);

-- Step 2: Insert two order items
INSERT INTO order_items VALUES
  (5016, 1011, 206, 1, 1299.00, 0),
  (5017, 1011, 208, 1, 599.00, 0);

-- Step 3: Update stock quantities for purchased products
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;

-- If any step above failed, run: ROLLBACK;
-- Otherwise commit all changes atomically:
COMMIT;
```

Screenshot:



```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO orders VALUES
-> (1011, 102, CURDATE(), 'Pending', 1598.00);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO order_items VALUES
-> (5016, 1011, 206, 1, 1299.00, 0),
-> (5017, 1011, 208, 1, 599.00, 0);
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```